

Zadanie 01 - Room Booking API

Założenia

Zbuduj proste REST API służące do zarządzania salami oraz ich rezerwacjami.

Na tym etapie **nie używamy bazy danych**.

Dane mogą być przechowywane w zwykłych tablicach JavaScript.

Przykładowa sala:

```
{
  "id": 1,
  "name": "A101",
  "capacity": 30
}
```

Przykładowa rezerwacja:

```
{
  "id": 1,
  "roomId": 1,
  "reservedBy": "Jan Kowalski",
  "participants": 20,
  "date": "2026-10-10",
  "status": "CONFIRMED"
}
```

Za cały zestaw można uzyskać maksymalnie **5 punktów**.

Każde zadanie jest warte **1 punkt**.

Zadanie 1 - Rooms API - 1 pkt

Utwórz endpoint:

```
GET /api/rooms
```

który zwraca wszystkie dostępne sale.

Następnie utwórz:

```
GET /api/rooms/:id
```

który zwraca pojedynczą salę.

Wymagania

- przygotuj minimum 4 przykładowe sale,
- każda sala posiada `id`, `name` oraz `capacity`,
- istniejące ID zwraca odpowiednią salę,
- nieistniejące ID zwraca `404 Not Found`.

Przykład:

```
GET /api/rooms/1
```

Response:

```
{  
  "id": 1,
```

```
"name": "A101",  
"capacity": 30  
}
```

Zadanie 2 - Tworzenie rezerwacji - 1 pkt

Utwórz endpoint:

```
POST /api/reservations
```

Przykładowy request:

```
{  
  "roomId": 1,  
  "reservedBy": "Jan Kowalski",  
  "participants": 20,  
  "date": "2026-10-10"  
}
```

Backend powinien automatycznie dodać:

- `id`,
- `status` `CONFIRMED`.

Przykładowy response:

```
{  
  "id": 1,  
  "roomId": 1,  
  "reservedBy": "Jan Kowalski",  
  "participants": 20,
```

```
"date": "2026-10-10",  
"status": "CONFIRMED"  
}
```

Poprawnie utworzona rezerwacja powinna zwracać:

201 Created

Validation

Sprawdź, czy:

- `roomId` zostało podane,
- wskazana sala istnieje,
- `reservedBy` zostało podane,
- `participants` jest liczbą większą od 0,
- `date` zostało podane.

Jeżeli dane są niepoprawne, zwróć:

400 Bad Request

Jeżeli sala nie istnieje, zwróć:

404 Not Found

Zadanie 3 - Business rule: pojemność sali - 1 pkt

Dodaj prostą regułę biznesową.

Liczba uczestników rezerwacji nie może być większa niż pojemność wybranej sali.

Przykład:

Sala:

```
{  
  "id": 1,  
  "name": "A101",  
  "capacity": 30  
}
```

Request:

```
{  
  "roomId": 1,  
  "reservedBy": "Jan Kowalski",  
  "participants": 40,  
  "date": "2026-10-10"  
}
```

Taka rezerwacja powinna zostać odrzucona.

Zwróć:

```
409 Conflict
```

Przykładowe body:

```
{
```

```
"message": "Room capacity exceeded"
}
```

Dla `participants = 30` rezerwacja powinna zostać zaakceptowana.

Zadanie 4 - Pobieranie i filtrowanie rezerwacji - 1 pkt

Utwórz endpoint:

```
GET /api/reservations
```

Bez dodatkowych parametrów endpoint powinien zwracać wszystkie rezerwacje.

Następnie dodaj możliwość filtrowania po `roomId`.

Przykład:

```
GET /api/reservations?roomId=1
```

powinien zwrócić tylko rezerwacje dotyczące sali o ID `1`.

Wymagania

- brak `roomId` → wszystkie rezerwacje,
- podane `roomId` → tylko rezerwacje wybranej sali,
- jeżeli nie znaleziono żadnych rezerwacji, zwróć pustą tablicę:

```
[]
```

Zadanie 5 - Anulowanie rezerwacji - 1 pkt

Utwórz endpoint:

```
PATCH /api/reservations/:id/cancel
```

Endpoint powinien zmieniać status rezerwacji:

```
CONFIRMED
```

na:

```
CANCELLED
```

Wymagania

- istniejąca rezerwacja → zmiana statusu i 200 OK,
- nieistniejąca rezerwacja → 404 Not Found,
- jeżeli rezerwacja ma już status CANCELLED, endpoint może zwrócić tę rezerwację bez ponownej zmiany.

Przykładowy response:

```
{
  "id": 1,
  "roomId": 1,
  "reservedBy": "Jan Kowalski",
```

```
"participants": 20,  
"date": "2026-10-10",  
"status": "CANCELLED"  
}
```

Punktacja

Zadanie	Punkty
Rooms API	1 pkt
Tworzenie rezerwacji + validation	1 pkt
Reguła pojemności sali	1 pkt
Pobieranie i filtrowanie rezerwacji	1 pkt
Anulowanie rezerwacji	1 pkt
Razem	5 pkt

Co oddać?

Przygotuj sprawozdanie w formacie PDF.

Powinno ono zawierać:

- fragmenty najważniejszego kodu,
- screenshoty potwierdzające działanie endpointów,
- przykłady poprawnych requestów,
- przynajmniej jeden przykład obsłużonego błędu,
- krótki opis rozwiązania.

Nie ma potrzeby umieszczania screenshotu całego kodu.

Najważniejsze jest pokazanie, że API działa zgodnie z wymaganiami.